



## Chương 2.

# GIỚI THIỆU VỀ UML

# Tổng quan về UML



- **UML (Unified Model Language):** là ngôn ngữ mô hình hóa thống nhất bao gồm những ký hiệu hình học, dùng mô hình hóa trong phương pháp hướng đối tượng (OOAD – Object Oriented Analysis and Design)
- **UML dùng để:**
  - Trực quan hóa (Visualizing)
  - Đặc tả (Specifying)
  - Xây dựng (Constructing)
  - Lập tài liệu (Documenting)

# Tổng quan về UML (tt)



- UML còn được dùng làm công cụ giao tiếp giữa người dùng, nhà phân tích, nhà thiết kế và nhà phát triển phần mềm
- Được duy trì và phát triển bởi OMG (Object Management Group), do **Jacobson, Booch, Rumbaugh** sáng lập. Ngoài ra còn có hàng trăm các tập đoàn lớn khác bảo trợ phát triển

# Tổng quan về UML (tt)



## ■ Grady Booch

- Làm việc cho ADA, sau đó là US Air Force Academy
- Giám đốc khoa học của RSC

## ■ Jame Rumbaugh

- Nhân vật đứng đầu của General Electric
- Tác giả của cuốn Object Modelling Technique

## ■ Ivar Jacobson

- Làm cho Ericson, sau đó sở hữu công ty Objective System ở Thụy Điển
- Là cha đẻ của Use Case

# Trực quan hóa (Visualizing)



- Sử dụng các **Sơ đồ** và **ký hiệu chuẩn** để biểu diễn các khái niệm trừu tượng của hệ thống
- Cung cấp một ngôn ngữ hình ảnh chung, giúp các nhà phát triển, nhà phân tích và các bên liên quan giao tiếp hiệu quả về thiết kế, cấu trúc, hành vi cũng như các khía cạnh khác của hệ thống
  - Làm rõ khái niệm phức tạp
  - Hỗ trợ thiết kế và phân tích
  - Tài liệu hóa hệ thống

# Đặc tả (Specifying)



- UML giúp xây dựng các mô hình chính xác, đầy đủ và đặc tả các khía cạnh của hệ thống:
  - Yêu cầu chức năng
  - Cấu trúc hệ thống
  - Hành vi hệ thống
  - Ràng buộc trong luồng công việc

# Xây dựng (Constructing)



- Quá trình thực hiện hoặc phát triển hệ thống phần mềm dựa trên các mô hình và Sơ đồ mà UML cung cấp:
  - Chuyển đổi mô hình thành mã nguồn
  - Triển khai hệ thống
  - Kiểm thử và tinh chỉnh

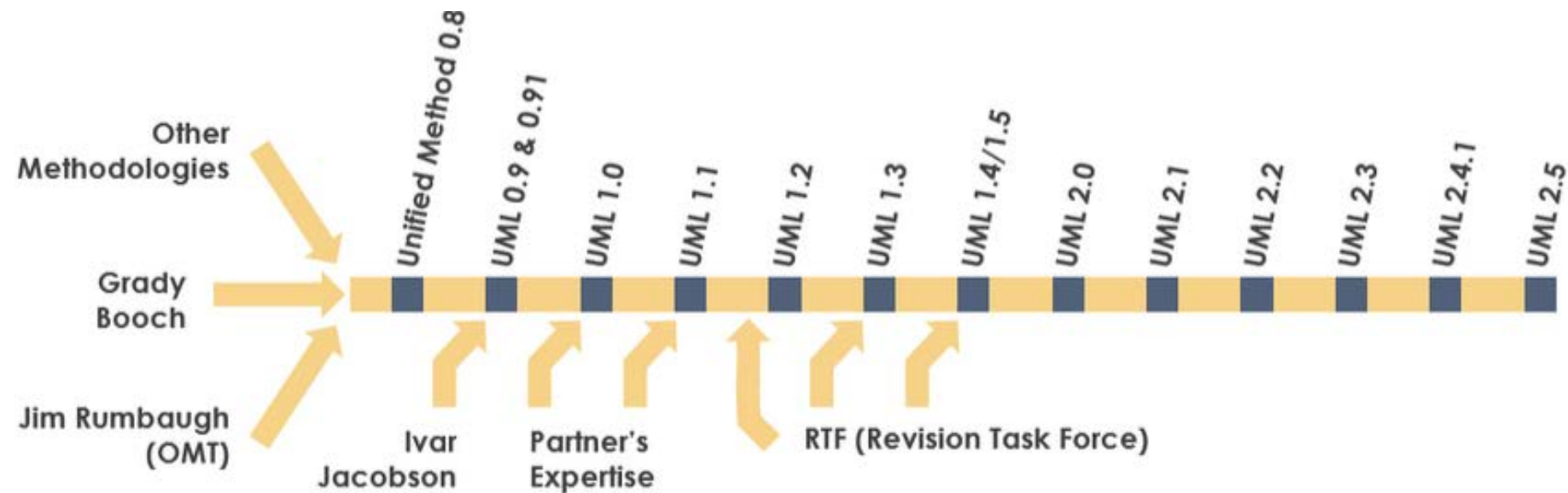
# Viết tài liệu (Documenting)



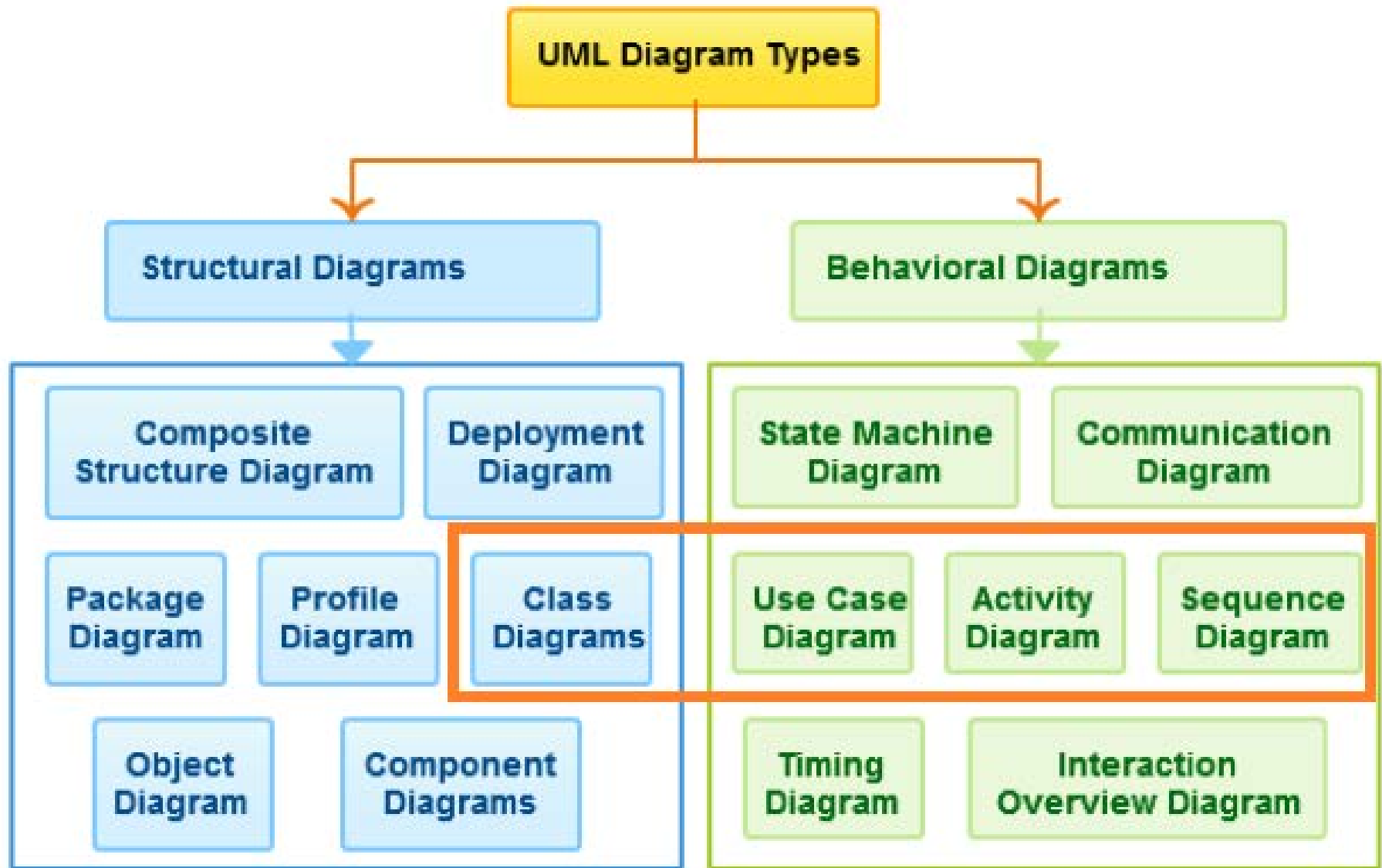
- Quá trình tạo ra các tài liệu mô tả hệ thống một cách chi tiết, dựa trên các mô hình và thông tin từ các giai đoạn phân tích, thiết kế, và xây dựng
- Đóng vai trò quan trọng trong việc giao tiếp giữa các bên liên quan, hỗ trợ phát triển, bảo trì, và mở rộng hệ thống trong tương lai
  - Giao tiếp
  - Hướng dẫn
  - Lưu trữ



# Lịch sử phát triển



# Các sơ đồ của UML



# Các sơ đồ của UML (tt)



## UML 1.x có 9 loại sơ đồ:

1. Sơ đồ Use Case (Use Case Diagram)
2. Sơ đồ lớp (Class Diagram)
3. Sơ đồ đối tượng (Object Diagram)
4. Sơ đồ trạng thái (State Diagram)
5. Sơ đồ trình tự (Sequence Diagram)
6. Sơ đồ cộng tác (Collaboration Diagram)
7. Sơ đồ hoạt động (Activity Diagram)
8. Sơ đồ thành phần (Component Diagram)
9. Sơ đồ triển khai (Deployment Diagram)

# Các sơ đồ của UML (tt)



## UML 2.5 có 14 loại Sơ đồ:

### Thêm các sơ đồ:

- Sơ đồ gói (Package Diagram)
- Sơ đồ cấu trúc đa hợp (Composite Structure Diagram)
- Sơ đồ tương tác tổng quát (interaction overview Diagram)
- Sơ đồ thời gian (Timing Diagram)

### Ngoài ra:

- Sơ đồ trạng thái (state chart diagram) đổi tên thành sơ đồ máy trạng thái (state machine diagram)
- Sơ đồ cộng tác (collaboration diagram) đổi tên lại là sơ đồ giao tiếp (communication diagram)

# Các sơ đồ của UML (tt)



Các sơ đồ được phân thành 2 nhóm:

- Nhóm sơ đồ **cấu trúc** (Structure Diagram):
  - Sơ đồ lớp (Class Diagram)
  - Sơ đồ đối tượng (Object Diagram)
  - Sơ đồ thành phần (Component Diagram)
  - Sơ đồ cấu trúc đa hợp (Composite Structure Diagram)
  - Sơ đồ triển khai (Deployment Diagram)
  - Sơ đồ gói (Package Diagram)

# Các sơ đồ của UML (tt)



## ▪ Nhóm sơ đồ **hành vi** (Behaviour Diagram):

- Sơ đồ Use Case (Use Case Diagram)
- Sơ đồ hoạt động (Activity Diagram)
- Sơ đồ máy trạng thái (State Machine Diagram)
- Nhóm sơ đồ tương tác (Interaction Diagram):
  - ✓ Sơ đồ trình tự (Sequence Diagram)
  - ✓ Sơ đồ giao tiếp (Communication Diagram)
  - ✓ Sơ đồ tương tác tổng quát (interaction overview Diagram)
  - ✓ Sơ đồ thời gian (Timing Diagram)

- Góc nhìn chỉ ra những khía cạnh khác nhau của hệ thống cần phải được mô hình hóa
- Một góc nhìn không phải là một bản vẽ cụ thể, mà là một sự trừu tượng hóa bao gồm một loạt các sơ đồ khác nhau
- Chỉ qua việc định nghĩa của một loạt các góc nhìn khác nhau, mỗi góc nhìn chỉ ra một khía cạnh riêng biệt của hệ thống, người ta mới có thể tạo dựng nên một bức tranh hoàn thiện về hệ thống

# Góc nhìn (View) (tt)



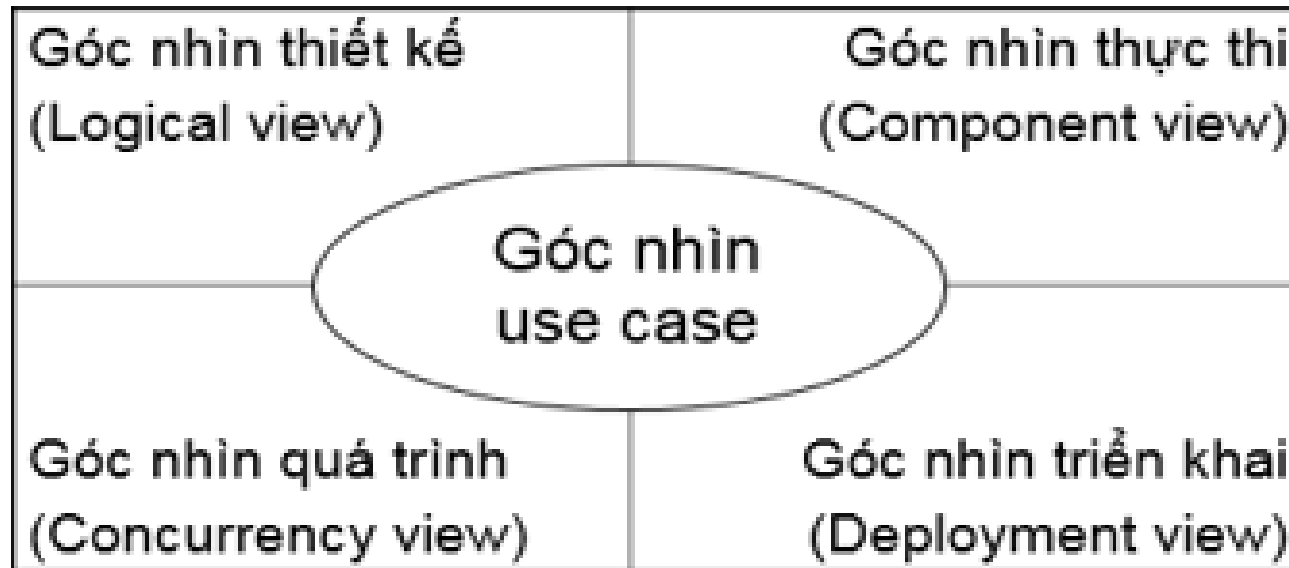
- Góc nhìn (có một số tài liệu gọi là khung nhìn, hay hướng nhìn) là một khái niệm trong UML, chứ không phải là một thành phần biểu diễn có thể nhìn thấy được như Use Case, sơ đồ, lớp...
- Trong đời thường, khi quan sát một vật thể phức tạp ta phải nhìn từ nhiều hướng khác nhau. Khi biểu diễn các vật đó trên giấy cũng không thể chỉ biểu diễn trong một bản vẽ duy nhất mà phải dùng nhiều bản vẽ, mỗi bản vẽ biểu diễn vật từ một góc nhìn



# Các khung nhìn (View) của UML

**Người dùng**  
Chức năng

**Lập trình viên**  
Quản trị phần mềm



**Quản trị viên tích hợp hệ thống**  
Hiệu năng  
Tính co giãn  
Thông lượng

**Thiết kế viên hệ thống**  
Hình thái hệ thống  
Chuyển giao, cài đặt  
Truyền thông

# Góc nhìn Use Case

- Là góc nhìn từ ngoài vào hệ thống
- Đây là góc nhìn dành cho khách hàng, nhà thiết kế, nhà phát triển, người kiểm thử. Mô tả các chức năng mong đợi mà hệ thống phải đáp ứng cho người dùng
- Được mô tả qua các sơ đồ Use Case (Use Case Diagram) và thỉnh thoảng cả các sơ đồ hoạt động (Activity Diagram)

# Góc nhìn Use Case (tt)



- Góc nhìn Use Case mang tính trung tâm. Hệ thống phải cung cấp các chức năng mô tả trong hướng nhìn này → có ảnh hưởng đến tất cả các hướng nhìn khác.
- Góc nhìn này còn được sử dụng để thẩm tra (verify) hệ thống xem có đúng với mong đợi của khách hàng (*"Đây có phải là thứ bạn muốn?"*) cũng như có đúng với hệ thống đã hoàn thành (*"Hệ thống có hoạt động đúng như đã đặc tả?"*)

- Còn gọi là góc nhìn logic (logical view)
- Chủ yếu dùng cho các nhà thiết kế và nhà phát triển
- Đây là góc nhìn vào bên trong hệ thống. Nó chỉ ra chức năng sẽ được thiết kế bên trong hệ thống như thế nào
- Tập trung vào chức năng mà hệ thống cung cấp cho người dùng cuối, thường được mô tả bằng các Sơ đồ lớp hoặc Sơ đồ đối tượng → **Hệ thống làm cái gì?**

# Góc nhìn thiết kế (tt)



- Cấu trúc tĩnh được mô tả bằng các sơ đồ lớp (class diagram) và sơ đồ đối tượng (object diagram)
- Cấu trúc động được mô tả trong các sơ đồ trạng thái (state chart diagram), sơ đồ trình tự (sequence diagram), sơ đồ tương tác (collaboration diagram), sơ đồ hoạt động (activity diagram)

- Còn gọi là góc nhìn thành phần (Component view)
- Là góc nhìn đối với dạng phát hành của hệ thống vật lý, bao gồm các thành phần và tập tin độc lập, được lắp ráp theo các cách để tạo ra hệ thống chạy được
- Thành phần ở đây là các module thuộc nhiều loại khác nhau, sẽ được chỉ ra trong sơ đồ cùng với cấu trúc cũng như sự phụ thuộc của chúng
- Thường được sử dụng cho nhà phát triển và thường bao gồm nhiều sơ đồ thành phần

**➔ Hệ thống được xây dựng ra sao**

- Góc nhìn triển khai (Deployment View) chỉ cho chúng ta sơ đồ triển khai về mặt vật lý của hệ thống
- Góc nhìn triển khai dành cho các nhà phát triển, người tích hợp cũng như người thử nghiệm hệ thống và được thể hiện bằng các sơ đồ triển khai (deployment diagram)
- Góc nhìn này cũng bao gồm sự ánh xạ các thành phần của hệ thống vào cấu trúc vật lý

**➔ Hệ thống được đặt ở đâu**

- Còn gọi là góc nhìn song hành (Concurrency View)
- Phản ánh các lộ trình điều khiển, các quá trình (process) thực hiện, nó cho thấy sự hoạt động song hành hay đồng bộ của hệ thống
- Góc nhìn này dành cho nhà phát triển và người tích hợp hệ thống

**➔ Hệ thống hoạt động thế nào**



## Giai đoạn nghiên cứu sơ bộ:

- UML đưa ra khái niệm Use Case để nắm bắt các yêu cầu của người sử dụng
- Sơ đồ Use Case dùng để mô tả mối quan hệ cũng như sự giao tiếp với hệ thống
- Trong sơ đồ Use Case, các tác nhân (Actor) bên ngoài quan tâm đến hệ thống sẽ được mô hình song song với chức năng mà họ đòi hỏi từ hệ thống
- Mỗi Use Case trong tài liệu sẽ đặc tả yêu cầu: người dùng chờ đợi điều gì từ hệ thống mà không để ý đến chức năng này sẽ được thực thi ra sao

## Giai đoạn phân tích:

- Giai đoạn phân tích quan tâm đến việc trừu tượng hóa
- Phân tích viên tìm ra các lớp thành phần và mối quan hệ giữa chúng → mô tả bằng sơ đồ lớp
- Sự cộng tác giữa các lớp nhằm thực hiện các Use Case cũng sẽ được mô tả nhờ vào các mô hình động (dynamic models) của UML
- Trong giai đoạn phân tích, chỉ các lớp tồn tại trong phạm vi nghiệp vụ cần giải quyết (lớp phân tích) là được mô hình hóa. Các lớp kỹ thuật cũng như giải pháp trong hệ thống phần mềm, ví dụ các lớp giao diện người dùng, v.v..., chưa cần quan tâm trong giai đoạn này

## Giai đoạn thiết kế:

- Kết quả của giai đoạn phân tích sẽ được mở rộng thành các giải pháp kỹ thuật
- Các lớp thiết kế được bổ sung để tạo thành hạ tầng kỹ thuật: giao diện người dùng, các chức năng để giao tiếp với các hệ thống khác, v,v....
- Giai đoạn thiết kế sẽ đưa ra kết quả là bản đặc tả chi tiết (specification) dùng để xây dựng hệ thống

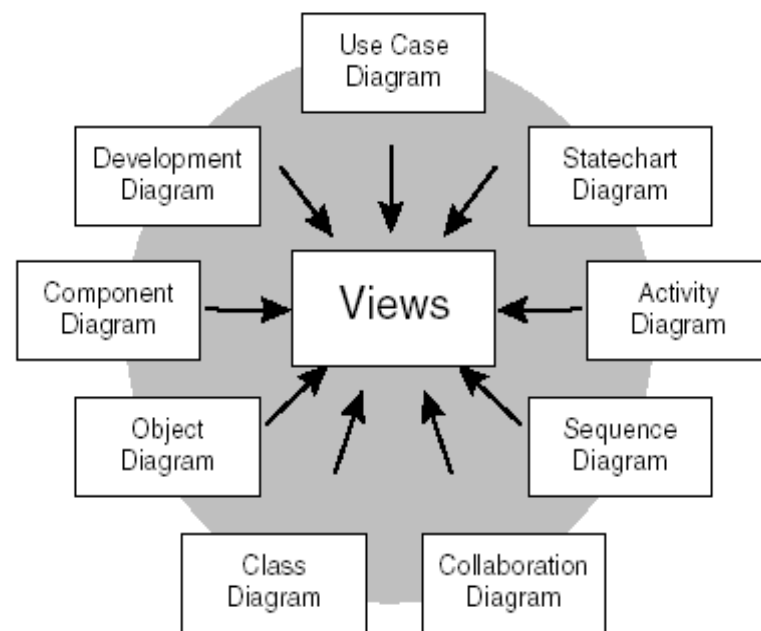
## Giai đoạn xây dựng:

- Trong giai đoạn xây dựng (lập trình), các lớp của giai đoạn thiết kế sẽ được chuyển thành những dòng code cụ thể trong một ngôn ngữ lập trình hướng đối tượng cụ thể
- Khi tạo ra các mô hình phân tích và thiết kế trong UML, cần tránh việc ngay lập tức biến đổi các mô hình này thành các dòng code
- Trong các giai đoạn trước, mô hình được sử dụng để hiểu rõ ràng hệ thống, dễ giao tiếp và tạo nên cấu trúc của hệ thống. Nếu vội vàng viết code có thể sẽ gây trở ngại cho việc xây dựng mô hình chính xác và đầy đủ

## Giai đoạn kiểm thử:

- Một hệ thống thường được thử nghiệm qua nhiều giai đoạn và với nhiều nhóm thử nghiệm khác nhau
- Các nhóm sử dụng các loại sơ đồ UML khác nhau làm nền tảng cho công việc của mình:
  - Kiểm thử đơn vị (unit test) sử dụng sơ đồ lớp và đặc tả lớp
  - Kiểm thử nghiệm tích hợp (integration test) thường sử dụng sơ đồ thành phần (component diagram) và sơ đồ cộng tác (collaboration diagram)
  - Kiểm thử hệ thống (system test) sử dụng sơ đồ Use Case (Use Case diagram) để đảm bảo hệ thống hoạt động đúng như đã được định nghĩa từ ban đầu

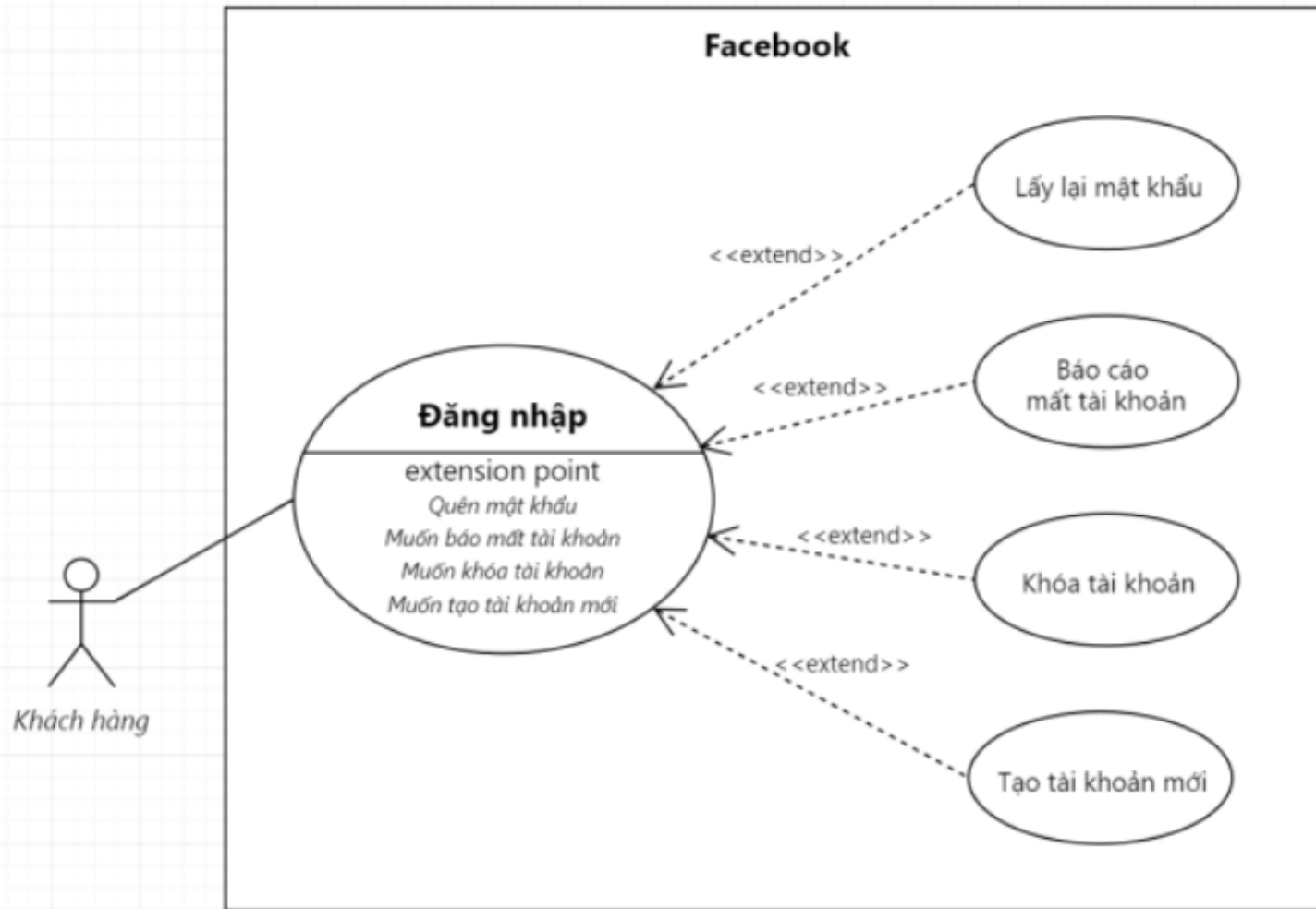
- Các sơ đồ góp phần **chi tiết hóa** hệ thống
  - Dựa vào các sơ đồ, các nhà phát triển có thể **giao tiếp với nhau** và **giải quyết các vấn đề phức tạp** của hệ thống
  - Các sơ đồ kết nối lại với nhau tạo nên một cái nhìn tổng quát về hệ thống
  - Mỗi sơ đồ mô tả một khía cạnh khác nhau của hệ thống
- ➔ **Sơ đồ Use Case, hoạt động, tuần tự, và sơ đồ lớp thường được sử dụng nhất**



# Use Case Diagram

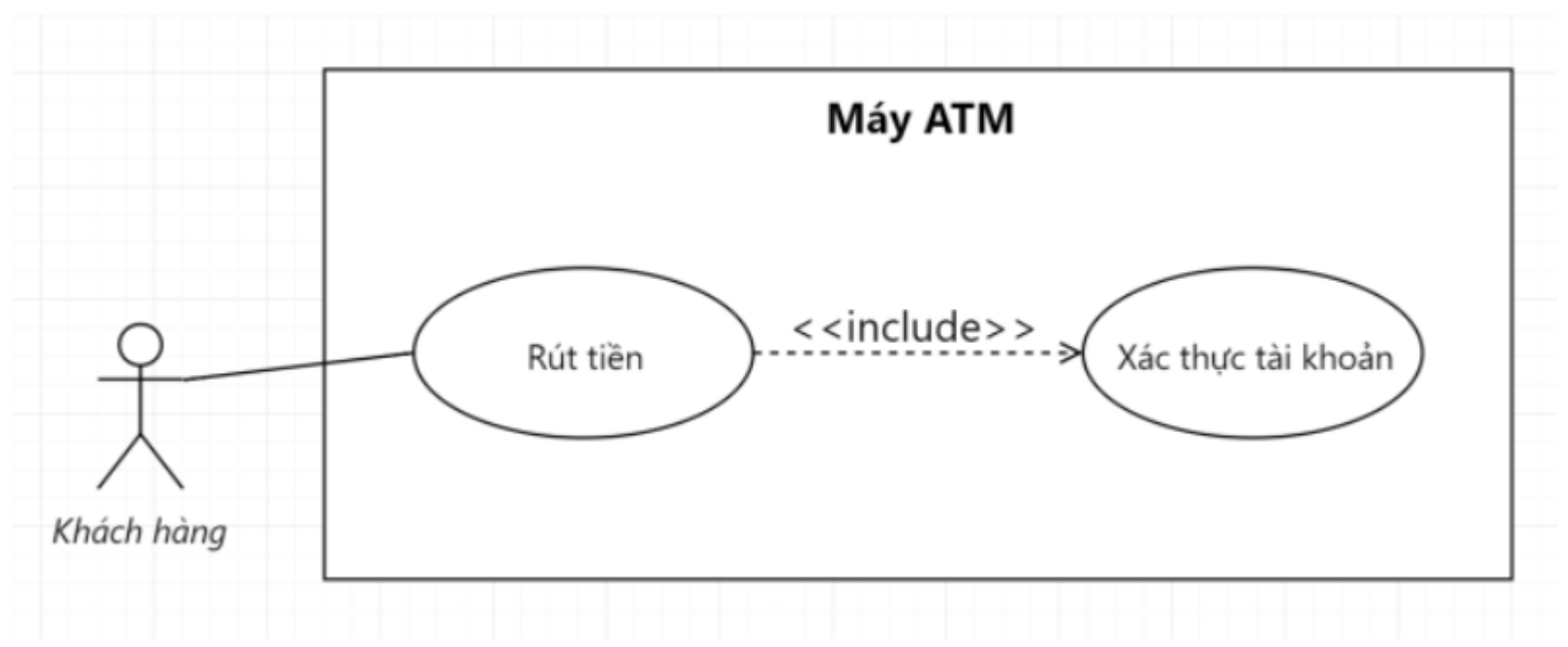
- Sơ đồ Use Case chỉ ra các tác nhân ngoài và mối liên hệ giữa chúng với các Use Case mà hệ thống cung cấp
- Một Use Case là diễn tả một chức năng mà hệ thống có khả năng cung cấp
- Mỗi Use Case có thể được đặc tả bằng một văn bản, hay bằng một sơ đồ hoạt động
- Các Use Case được miêu tả **duy nhất** theo hướng nhìn từ ngoài vào của các tác nhân (hành vi của hệ thống theo như sự mong đợi của người sử dụng)
- Các Use Case chỉ đề cập hệ thống làm gì **(WHAT)** chứ không mô tả chức năng được cung cấp sẽ hoạt động nội bộ bên trong hệ thống ra sao **(HOW)**

# Use Case Diagram (tt)

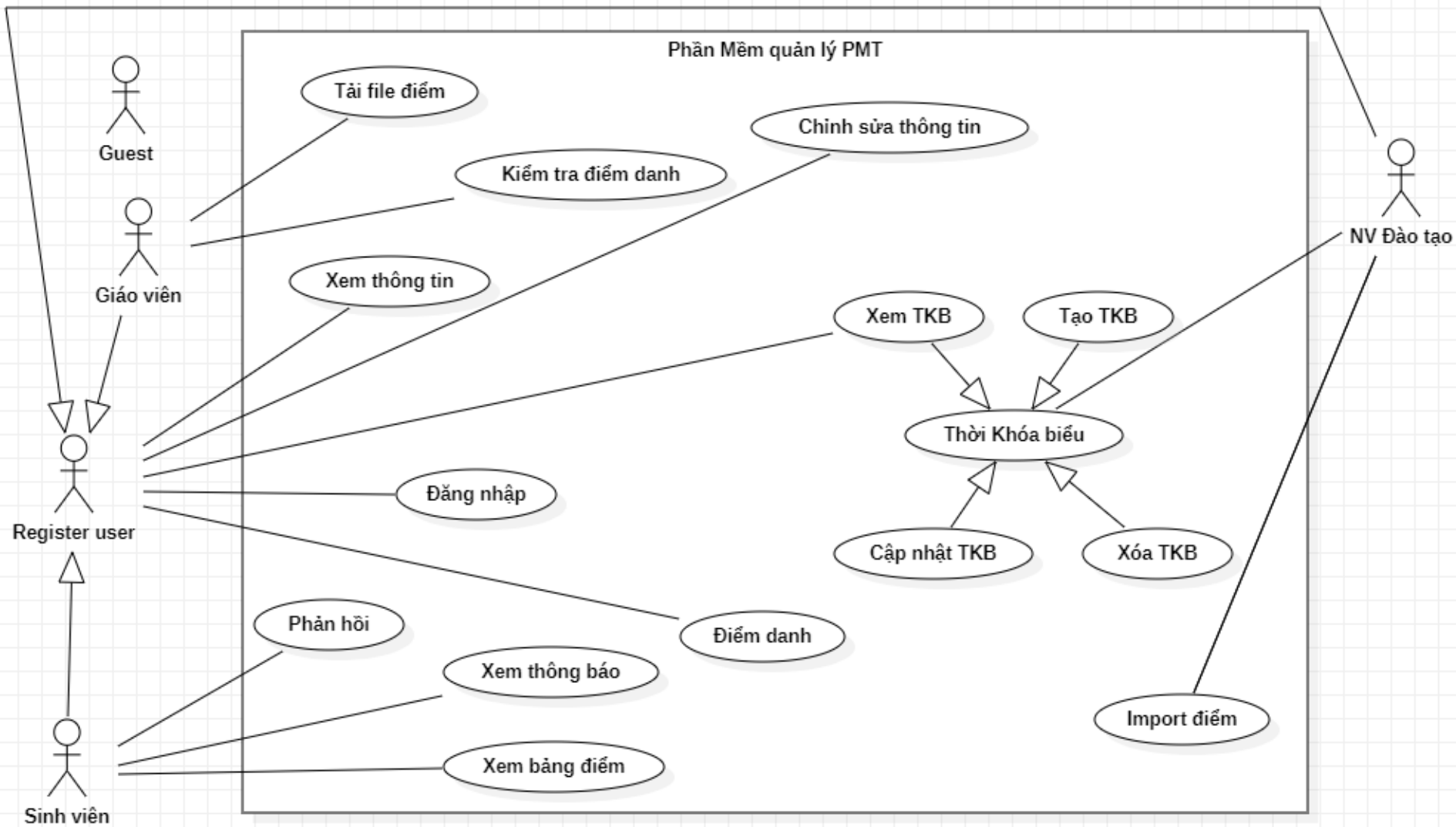




# Use Case Diagram (tt)



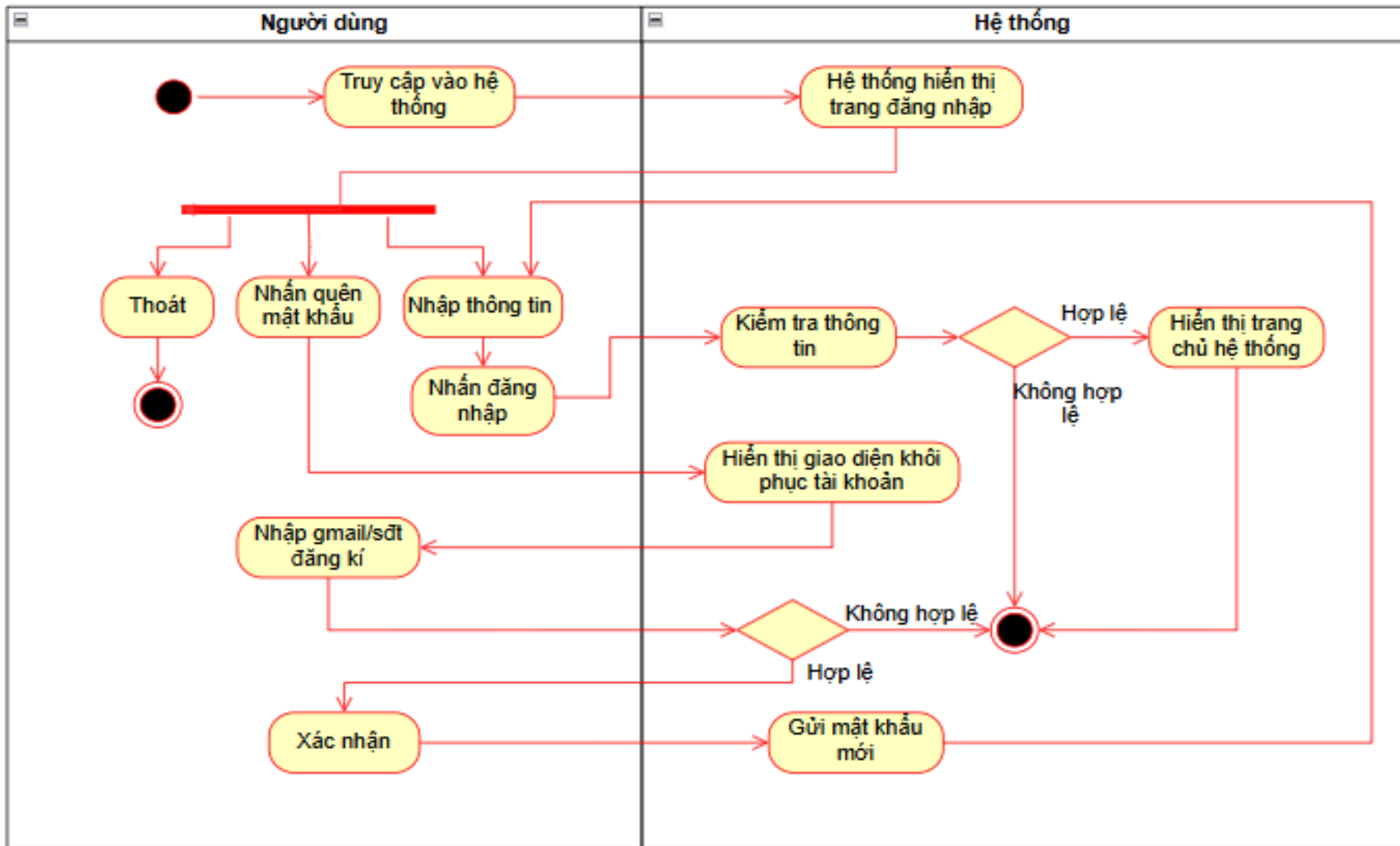
# Use Case Diagram (tt)



# Activity Diagram

- Sơ đồ hoạt động chỉ ra luồng dịch chuyển từ hành động này sang hành động khác
- Thường sử dụng để mô tả các hoạt động được thực hiện trong một thủ tục, một Use Case. Cũng có thể dùng để mô tả các dòng chảy giữa các Use Case
- Sơ đồ hoạt động bao gồm các trạng thái hành động, chứa đặc tả của một hoạt động cần phải được thực hiện (một hành động - action). Một trạng thái hành động sẽ qua đi khi hành động được thực hiện xong
- Sơ đồ còn có thể chỉ ra các quyết định, các điều kiện, cũng như phân thực thi song song của các trạng thái hành động
- Sơ đồ còn có thể chứa đặc tả cho các thông điệp được gửi đi/nhận về giữa các thành phần của hành động

# Activity Diagram (tt)



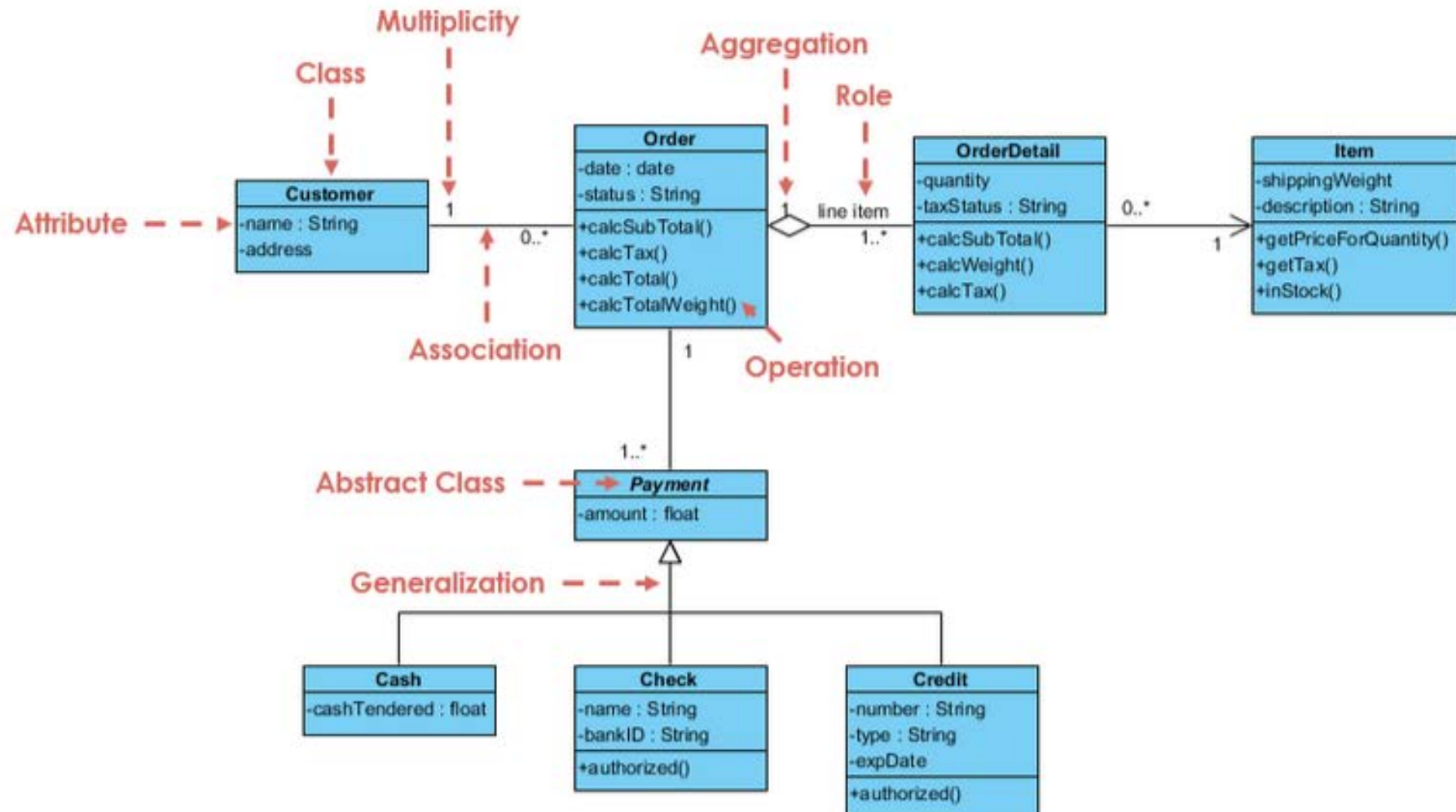
# Class Diagram



- Sơ đồ lớp chỉ ra cấu trúc tĩnh của các lớp trong hệ thống
- Vì cấu trúc được mô tả luôn luôn đúng tại bất kỳ thời điểm nào trong vòng đời hệ thống
- Lớp là đại diện cho các sự vật được xử lý trong hệ thống
- Các lớp có thể quan hệ với nhau theo nhiều cách
- Tất cả các mối quan hệ đó đều được thể hiện trong sơ đồ lớp, cùng với cấu trúc bên trong của mỗi lớp, gồm thuộc tính (attribute) và hành vi (operation)

# Class Diagram (tt)

## Class Diagram Example: Order System

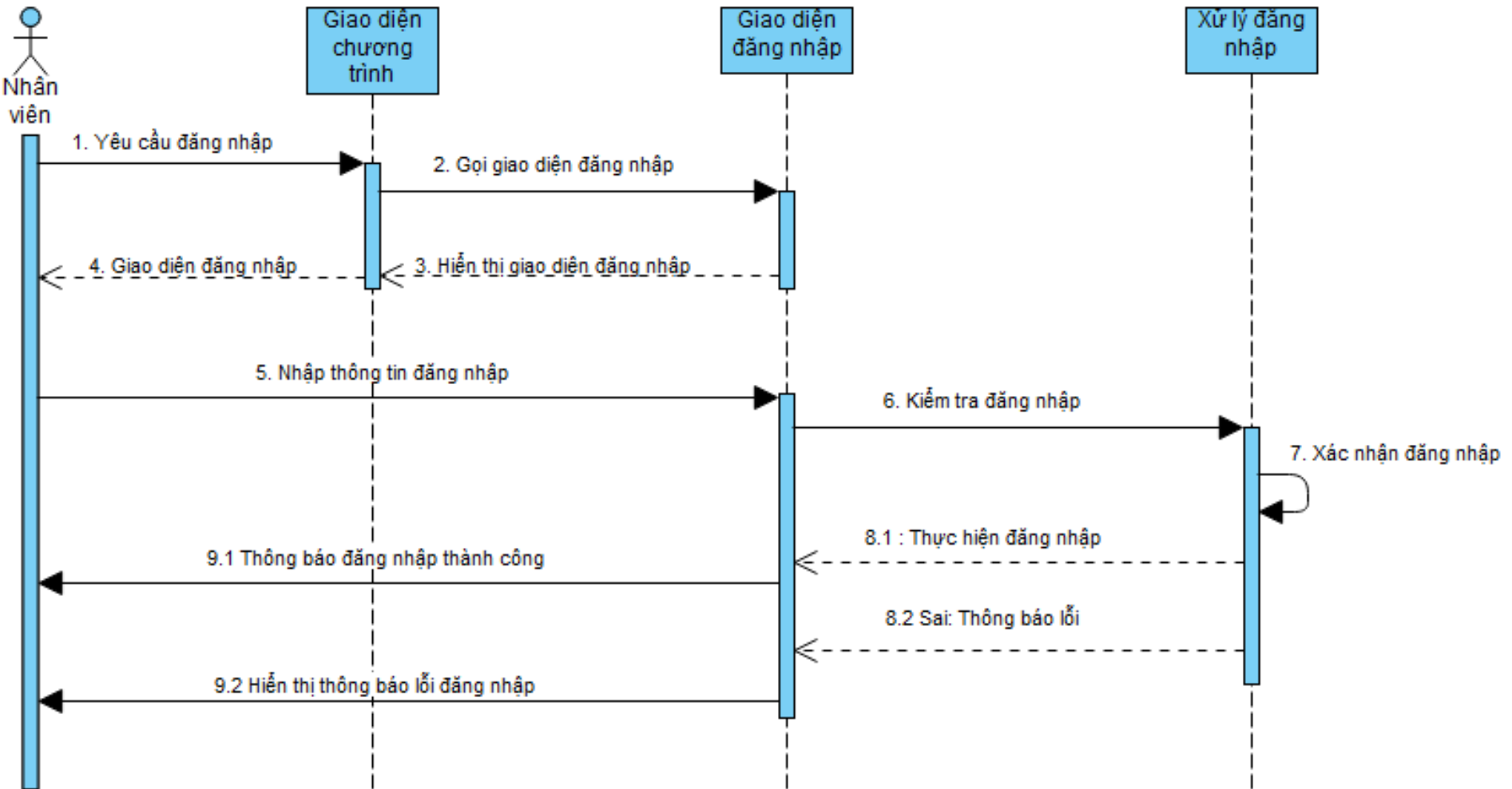


# Sequence Diagram



- Sơ đồ trình tự chỉ ra trình tự các thông điệp (message) được gửi giữa các đối tượng dọc theo trục thời gian
- Sơ đồ trình tự chứa một loạt các đối tượng được biểu diễn bằng các đường thẳng đứng
- Trục thời gian có hướng từ trên xuống dưới trong sơ đồ, và sơ đồ chỉ ra sự trao đổi thông điệp giữa các đối tượng khi thời gian trôi qua
- Các thông điệp được biểu diễn bằng các đường gạch ngang gắn liền với mũi tên (biểu thị thông điệp) nối liền giữa những đường thẳng đứng thể hiện đối tượng

# Sequence Diagram (tt)





# Communication diagram



- Tên cũ là sơ đồ cộng tác (Collaboration Diagram)
- Chỉ ra một sự cộng tác động, tương tự như sơ đồ trình tự
- Bên cạnh việc thể hiện sự trao đổi thông điệp (được gọi là tương tác), sơ đồ còn chỉ ra các đối tượng và quan hệ của chúng (được gọi là ngữ cảnh)
- Việc nên sử dụng sơ đồ trình tự hay sơ đồ giao tiếp thường được quyết định theo nguyên tắc chung sau:
  - Nếu thời gian hay trình tự là yếu tố quan trọng nhất cần phải nhấn mạnh thì hãy chọn sơ đồ trình tự
  - Nếu ngữ cảnh là yếu tố quan trọng hơn, chọn sơ đồ giao tiếp
- Trình tự tương tác giữa các đối tượng được thể hiện trong cả hai loại sơ đồ này

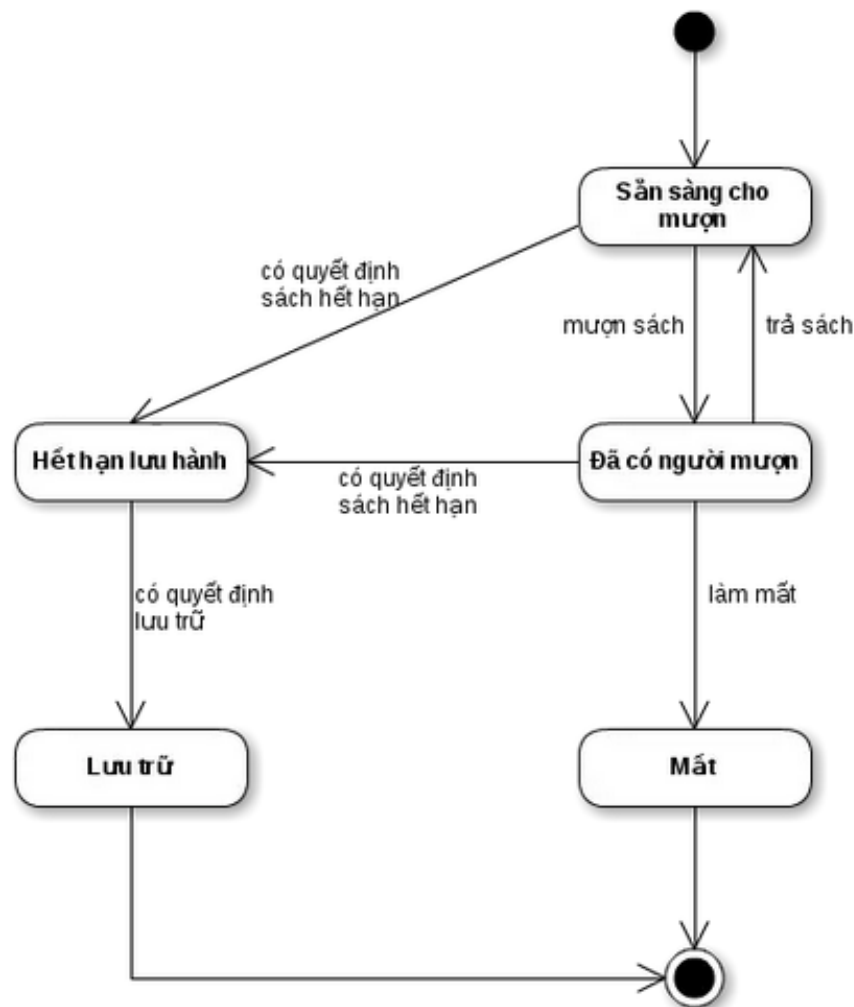
# State Machine Diagram



- Tên cũ là sơ đồ trạng thái (State chart Diagram)
- Sơ đồ trạng thái thường đặc tả của một lớp. Nó chỉ ra tất cả các trạng thái mà đối tượng của lớp này có thể có, và những sự kiện (event) nào sẽ gây ra sự thay đổi trạng thái
- Sự thay đổi trạng thái được gọi là dịch chuyển trạng thái (State Transition). Có thể có một hành động liên quan, xác định điều gì phải được thực hiện khi sự chuyển đổi trạng thái này diễn ra
- Sơ đồ trạng thái không cần phải vẽ cho tất cả các lớp, mà chỉ những lớp mà đối tượng của nó có khả năng ứng xử trước các sự kiện xảy đến tùy thuộc vào trạng thái hiện tại của nó

# State Machine Diagram (tt)

- Các trạng thái
- Sự chuyển tiếp
- Biến cố



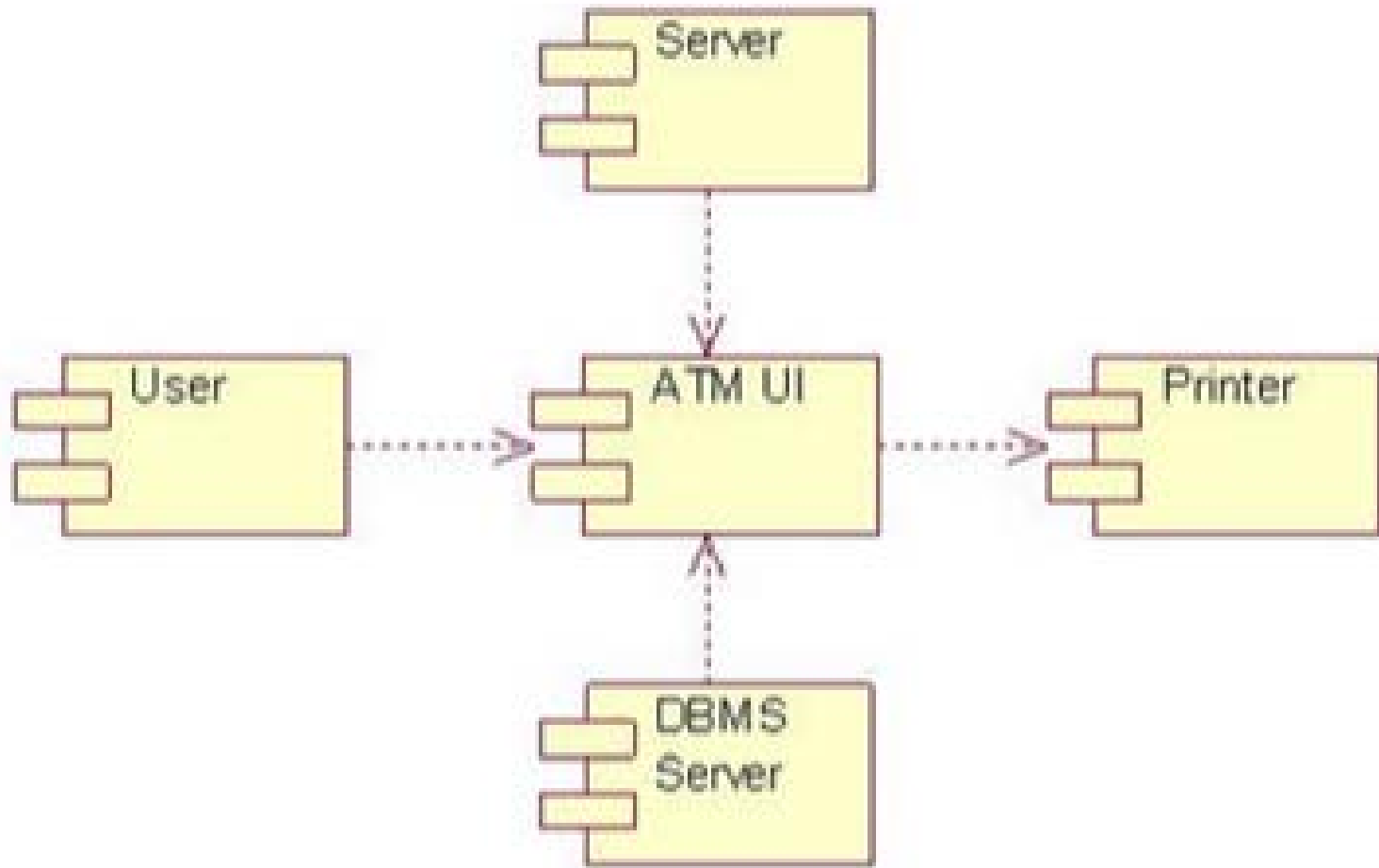
- Sơ đồ hoạt động *tập trung mô tả các hoạt động* và kết quả thu được từ việc *thay đổi trạng thái của đối tượng*
- Sơ đồ trạng thái *chỉ mô tả tập tất cả các trạng thái của một đối tượng* và những sự kiện dẫn tới sự thay đổi qua lại giữa các trạng thái đó

# Component Diagram



- Chỉ ra cấu trúc vật lý của chương trình dưới dạng các thành phần, và mối quan hệ giữa chúng
- Trình bày góc nhìn thực thi của hệ thống, được sử dụng trong việc lập trình
- Một thành phần có thể là một tập tin nguồn (source code), thành phần nhị phân (binary), thực thi (executable). Một thành phần chứa các thông tin về các lớp logic hoặc các lớp mà nó thi hành, như thế có nghĩa là nó tạo ra một ánh xạ từ góc nhìn logic vào góc nhìn thành phần
- Thành phần cũng có thể được mô tả với bất kỳ loại giao diện nào mà chúng bộc lộ, ví dụ giao diện OLE/ COM, có thể được nhóm lại với nhau thành từng gói (package)

# Component Diagram (tt)



# Deployment Diagram



- Sơ đồ triển khai trình bày kiến trúc vật lý của phần cứng cũng như phần mềm trong hệ thống. Nó chỉ ra từng máy tính cụ thể và từng trang thiết bị cụ thể (node) và sự nối kết giữa chúng
- Bên trong các nút mạng (node), các thành phần thực thi được (EXE) cũng như các đối tượng sẽ được xác định vị trí để chỉ ra những phần mềm nào sẽ được thực thi tại những nút mạng nào. Ta cũng có thể chỉ ra sự phụ thuộc giữa các thành phần
- Sơ đồ triển khai chỉ ra góc nhìn triển khai, mô tả kiến trúc vật lý thật sự của hệ thống. Đây là góc nhìn rất khác cách mô tả duy chức năng của góc nhìn Use Case

# Deployment Diagram (tt)

